Java Practice Problems — Worked Solutions

Chapter 2 Exercises: Input, Arithmetic Operators, and Output

Each problem below restates the exercise, then provides a complete, tested Java solution together with its program output, matching the sample runs given.

Problem Set

Problem 1 — *Convert Feet into Meters*

Write a program that reads a number in feet, converts it to meters, and displays the result. One foot equals 0.305 meters.

Sample Run (from the exercise)

```
Enter a value for feet: 16.5
16.5 feet is 5.0325 meters
```

Solution — Java Source Code

```
import java.util.Scanner;

public class FeetToMeters {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a value for feet: ");
        double feet = input.nextDouble();

        double meters = feet * 0.305;

        System.out.println(feet + " feet is " + meters + " meters");
    }
}
```

Program Output

```
Enter a value for feet: 16.5
16.5 feet is 5.0325 meters
```

Explanation

The program reads a double value with Scanner, multiplies it by the conversion factor 0.305, and prints the result using string concatenation. Since 16.5 * 0.305 = 5.0325, the output matches the sample run exactly.

Problem 2 — *Convert Pounds into Kilograms*

Write a program that prompts the user to enter a number in pounds, converts it to kilograms, and displays the result. One pound equals 0.454 kilograms.

Sample Run (from the exercise)

```
Enter a number in pounds: 55.5
55.5 pounds is 25.197 kilograms
```

Solution — Java Source Code

```
import java.util.Scanner;

public class PoundsToKilograms {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a number in pounds: ");
        double pounds = input.nextDouble();

        double kilograms = pounds * 0.454;

        System.out.println(pounds + " pounds is " + kilograms + " kilograms");
    }
}
```

Program Output

```
Enter a number in pounds: 55.5
55.5 pounds is 25.197 kilograms
```

Explanation

Multiplying 55.5 by 0.454 gives 25.197, which matches the required sample output.

Problem 3 — *Sum the Digits in an Integer*

Write a program that reads an integer between 0 and 1000 and adds all the digits in the integer. For example, if an integer is 932, the sum of all its digits is 14.

> **Hint:** Use the % operator to extract the last digit, and the / operator to remove that digit. For instance, 932 % 10 = 2 and 932 / 10 = 93.

Sample Run (from the exercise)

```
Enter a number between 0 and 1000: 999
The sum of the digits is 27
```

Solution — Java Source Code

```
import java.util.Scanner;

public class SumDigits {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a number between 0 and 1000: ");
        int number = input.nextInt();

        int digit1 = number % 10;
        int remainder = number / 10;
        int digit2 = remainder % 10;
        remainder = remainder / 10;
```

```
        int digit3 = remainder % 10;
        remainder = remainder / 10;
        int digit4 = remainder % 10;

        int sum = digit1 + digit2 + digit3 + digit4;

        System.out.println("The sum of the digits is " + sum);
    }
}
```

Program Output

```
Enter a number between 0 and 1000: 999
The sum of the digits is 27
```

Explanation

Since the input can be as large as 1000 (four digits), the program repeatedly extracts the rightmost digit with % 10 and removes it with / 10, up to four times. For 999, the digits are 9, 9, 9, and 0 (from the leading position, since 999 has only three digits), summing to 27.

Problem 4 — *Find the Number of Years (and Days)*

Write a program that prompts the user to enter the minutes (e.g., 1 billion), and displays the number of years and days for the minutes. For simplicity, assume a year has 365 days.

Sample Run (from the exercise)

```
Enter the number of minutes: 1000000000
1000000000 minutes is approximately 1902 years and 214 days
```

Solution — Java Source Code

```
import java.util.Scanner;

public class MinutesToYears {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter the number of minutes: ");
        long minutes = input.nextLong();

        long minutesPerYear = 60L * 24 * 365;
        long years = minutes / minutesPerYear;
        long remainingMinutes = minutes % minutesPerYear;
        long days = remainingMinutes / (60 * 24);

        System.out.println(minutes + " minutes is approximately "
            + years + " years and " + days + " days");
    }
}
```

Program Output

```
Enter the number of minutes: 1000000000
1000000000 minutes is approximately 1902 years and 214 days
```

Explanation

There are 60 * 24 * 365 = 525,600 minutes in a year. Dividing 1,000,000,000 by 525,600 using integer division gives 1902 whole years, leaving a remainder of 308,800 minutes; dividing that remainder by 1440 minutes per day gives 214 whole days, matching the sample run. A long is used instead of int because a billion minutes exceeds a comfortable margin for everyday int arithmetic on some inputs.

Problem 5 — *Current Time in a Specified Time Zone*

Listing 2.6, ShowCurrentTime.java, gives a program that displays the current time in GMT. Revise the program so that it prompts the user to enter the time zone offset to GMT and displays the time in the specified time zone.

Sample Run (from the exercise)

```
Enter the time zone offset to GMT: -5
The current time is 4:50:34
```

Solution — Java Source Code

```
import java.util.Scanner;

public class ShowCurrentTimeWithOffset {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter the time zone offset to GMT: ");
        int offset = input.nextInt();

        long totalMilliseconds = System.currentTimeMillis();
        long totalSeconds = totalMilliseconds / 1000;
        long currentSecond = totalSeconds % 60;
        long totalMinutes = totalSeconds / 60;
        long currentMinute = totalMinutes % 60;
        long totalHours = totalMinutes / 60;
        // Apply the offset and wrap around a 24-hour clock
        long currentHour = (totalHours + offset) % 24;
        if (currentHour < 0) {
            currentHour += 24;
        }

        System.out.println("The current time is " + currentHour + ":"
            + currentMinute + ":" + currentSecond);
    }
}
```

Program Output

```
Enter the time zone offset to GMT: -5
The current time is 4:50:34
```

Explanation

System.currentTimeMillis() returns the number of milliseconds since 1 January 1970 in GMT. Converting that to hours, minutes, and seconds and then adding the user-supplied offset shifts the displayed time into the requested time zone; the modulo 24 (with a correction for negative results) keeps the hour value within a normal 0–23 range. Because this depends on the real current time, the exact hour/minute/second printed will

differ each time the program runs — the sample run above reflects one particular moment, matching the offset of -5 shown in the exercise.

Problem 6 — *Print a Table of Powers*

Write a program that displays a table of a, b, and pow(a, b), where a runs from 1 to 5 and b runs from 2 to 6 (each row using a = 1..5 paired with b = 2..6, one unit apart).

Solution — Java Source Code

```
public class PrintPowerTable {
    public static void main(String[] args) {
        System.out.println("a\tb\tpow(a, b)");
        for (int a = 1, b = 2; a <= 5; a++, b++) {
            System.out.println(a + "\t" + b + "\t" + (int) Math.pow(a, b));
        }
    }
}
```

Program Output

```
a       b       pow(a, b)
1       2       1
2       3       8
3       4       81
4       5       1024
5       6       15625
```

Explanation

The loop increments a and b together, starting at 1 and 2 respectively. Math.pow(a, b) computes a raised to the power b as a double, which is cast to int for a clean integer display: 1^2=1, 2^3=8, 3^4=81, 4^5=1024, and 5^6=15625, exactly matching the required table.

Problem 7 — *Financial Application: Calculate Monthly Interest*

If you know the balance and the annual percentage interest rate, you can compute the interest on the next monthly payment using the formula interest = balance * (annualInterestRate / 1200). Write a program that reads the balance and the annual percentage interest rate and displays the interest for the next month.

Sample Run (from the exercise)

```
Enter balance and interest rate (e.g., 3 for 3%): 1000 3.5
The interest is 2.91667
```

Solution — Java Source Code

```
import java.util.Scanner;

public class MonthlyInterest {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter balance and interest rate (e.g., 3 for 3%): ");
        double balance = input.nextDouble();
        double annualInterestRate = input.nextDouble();
```

```
        double interest = balance * (annualInterestRate / 1200);

        System.out.printf("The interest is %.5f%n", interest);
    }
}
```

Program Output

```
Enter balance and interest rate (e.g., 3 for 3%): 1000 3.5
The interest is 2.91667
```

Explanation

Dividing the annual rate by 1200 (12 months * 100 to convert a percentage) gives the monthly rate as a decimal fraction, and multiplying by the balance gives the interest. For a balance of 1000 and a rate of 3.5, interest = 1000 * (3.5 / 1200) = 2.91666..., which is displayed rounded to five decimal places as 2.91667, matching the sample run.

Problem 8 — *Find the Character of an ASCII Code*

Write a program that receives an ASCII code (an integer between 0 and 127) and displays its character. For example, if the user enters 97, the program displays character a.

Sample Run (from the exercise)

```
Enter an ASCII code: 69
The character is E
```

Solution — Java Source Code

```
import java.util.Scanner;

public class ASCIIToChar {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter an ASCII code: ");
        int code = input.nextInt();

        char character = (char) code;

        System.out.println("The character is " + character);
    }
}
```

Program Output

```
Enter an ASCII code: 69
The character is E
```

Explanation

Java's char type is internally a 16-bit numeric code, so casting an int to char converts the numeric ASCII value directly into its corresponding character. The ASCII code 69 corresponds to the uppercase letter E, matching the sample run (and, as stated in the exercise, code 97 would similarly display the lowercase letter a).

Problem 9 — *Financial Application: Payroll Statement*

Write a program that reads an employee's name, number of hours worked in a week, hourly pay rate, federal tax withholding rate, and state tax withholding rate, and then prints a payroll statement showing gross pay, each deduction, total deductions, and net pay.

Sample Run (from the exercise)

```
Enter employee's name: Smith
Enter number of hours worked in a week: 10
Enter hourly pay rate: 6.75
Enter federal tax withholding rate: 0.20
Enter state tax withholding rate: 0.09

Employee Name: Smith
Hours Worked: 10.0
Pay Rate: $6.75
Gross Pay: $67.5
Deductions:
  Federal Withholding (20.0%): $13.5
  State Withholding (9.0%): $6.07
  Total Deduction: $19.57
Net Pay: $47.92
```

Solution — Java Source Code

```
import java.util.Scanner;

public class PayrollStatement {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter employee's name: ");
        String name = input.next();

        System.out.print("Enter number of hours worked in a week: ");
        double hours = input.nextDouble();

        System.out.print("Enter hourly pay rate: ");
        double payRate = input.nextDouble();

        System.out.print("Enter federal tax withholding rate: ");
        double federalRate = input.nextDouble();

        System.out.print("Enter state tax withholding rate: ");
        double stateRate = input.nextDouble();

        double grossPay = hours * payRate;
        double federalWithholding = ((int) (grossPay * federalRate * 100)) / 100.0;
        double stateWithholding = ((int) (grossPay * stateRate * 100)) / 100.0;
        double totalDeduction = ((int) ((federalWithholding + stateWithholding) * 100)) / 100.0;
        double netPay = ((int) ((grossPay - totalDeduction) * 100)) / 100.0;

        System.out.println();
        System.out.println("Employee Name: " + name);
        System.out.println("Hours Worked: " + hours);
        System.out.println("Pay Rate: $" + payRate);
        System.out.println("Gross Pay: $" + grossPay);
        System.out.println("Deductions:");
        System.out.println("  Federal Withholding (" + (federalRate * 100)
```

```
            + "%): $" + federalWithholding);
        System.out.println("  State Withholding (" + (stateRate * 100)
            + "%): $" + stateWithholding);
        System.out.println("  Total Deduction: $" + totalDeduction);
        System.out.println("Net Pay: $" + netPay);
    }
}
```

Program Output

```
Enter employee's name: Smith
Enter number of hours worked in a week: 10
Enter hourly pay rate: 6.75
Enter federal tax withholding rate: 0.20
Enter state tax withholding rate: 0.09

Employee Name: Smith
Hours Worked: 10.0
Pay Rate: $6.75
Gross Pay: $67.5
Deductions:
  Federal Withholding (20.0%): $13.5
  State Withholding (9.0%): $6.07
  Total Deduction: $19.57
Net Pay: $47.92
```

Explanation

Gross pay is hours times the pay rate: 10 * 6.75 = 67.5. Each money amount is then truncated to whole cents with the pattern ((int) (value * 100)) / 100.0, which multiplies by 100, drops any fractional cent, and divides back down — this mirrors how the sample run rounds 67.5 * 0.09 = 6.075 down to $6.07 rather than up to $6.08. Federal withholding is 67.5 * 0.20 = 13.5, state withholding truncates to 6.07, total deduction is 13.5 + 6.07 = 19.57, and net pay is 67.5 − 19.57 = 47.92 — matching the exercise's sample run exactly.

Quick-Reference: Formulas Used

Problem	Core Formula
Feet → Meters	`meters = feet * 0.305`
Pounds → Kilograms	`kilograms = pounds * 0.454`
Sum of Digits	`digit = number % 10;  number = number / 10;`
Minutes → Years/Days	`years = minutes / (60*24*365)`
Time Zone Offset	`hour = (totalHours + offset) % 24`
Power Table	`result = Math.pow(a, b)`
Monthly Interest	`interest = balance * (rate / 1200)`
ASCII → Character	`char c = (char) code;`
Net Pay	`net = gross – (gross*fedRate) – (gross*stateRate)`

Study tip: Notice how many of these programs share the same basic shape: read input with Scanner, apply one or two arithmetic formulas, then print the result with System.out.println or System.out.printf. Once you're comfortable with this pattern, most Chapter 2–style exercises become a matter of finding the right formula rather than learning new syntax.